

Week 2–3 — Integer Programs and the Binary-Variable Toolbox

OL7014 Integer Programming • Instructor: Prof. H. Park • Weeks of Aug 31 & Sep 7, 2026

1. Integer Programming (IP): add integrality — and lose convexity

one new line on last week's LP: $x, y \in \mathbb{Z}$

$X = P \cap \mathbb{Z}^n$ $P = \{x : Ax \leq b\}$ = linear container (last week's polyhedron), X = lattice points inside

read the symbols: $\mathbb{Z}$ = the integers, $\cap$ = intersection (points in *both* sets)

pure IP:

$$\begin{aligned} \max_x \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \mathbb{Z}^n \end{aligned}$$

binary program (BP) — every decision a yes/no switch:

$$\begin{aligned} \max_x \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \{0, 1\}^n \quad (\text{or } x \in \mathbb{B}^n) \end{aligned}$$

$x_j \in \{0, 1\}$ or $x_j \in \mathbb{B}$; $x_j \in \{0, 1\} \iff x_j \in \mathbb{Z}, 0 \leq x_j \leq 1$ — a BP is an IP with 0/1 bounds

mixed-integer LP (MILP) — integer x and continuous y mixed:

$$\begin{aligned} \max_{x, y} \quad & c^\top x + h^\top y \\ \text{s.t.} \quad & Ax + Gy \leq b \\ & x \in \mathbb{Z}_+^n, \quad y \geq 0 \end{aligned}$$

example: last week's toy with crates integer (x_1 = ammo, x_2 = rations), plus fuel y (liters — divisible):

$$\begin{aligned} \max \quad & 4x_1 + 5x_2 + 2y \\ \text{s.t.} \quad & x_1 + 3x_2 + y \leq 10 \\ & 3x_1 + x_2 + 2y \leq 10 \\ & x \in \mathbb{Z}_+^2, \quad y \geq 0 \end{aligned}$$

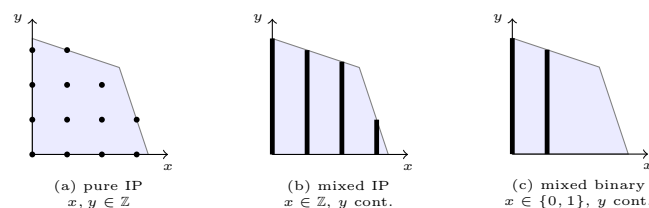
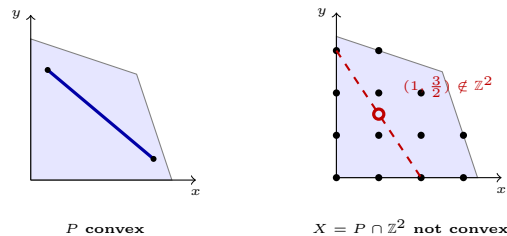
(c, A, b) we already know — the *only* new pieces are h and G :

$$c = \begin{pmatrix} 4 \\ 5 \\ 2 \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 3 \\ 3 & 1 \\ 0 & 0 \end{pmatrix}, \quad h = (2), \quad G = \begin{pmatrix} 1 \\ 2 \\ 0 \\ 0 \end{pmatrix}, \quad b = \begin{pmatrix} 10 \\ 10 \\ 0 \\ 0 \end{pmatrix}$$

(rows 3–4 are $x \geq 0$, exactly as last week; $y \geq 0$ comes with the form)

X is **not** convex — midpoint of two lattice points need not be one

KEY convention: x integer decisions, y continuous.



same P — what changes is *which points you may use*

takeaway one line, $\in \mathbb{Z}$, throws convexity away — the whole course is the bill for that line.

2. Why “relax and round” fails

LP relaxation: delete $x \in \mathbb{Z}^n \Rightarrow$ optimize over P instead of X

toy problem: LP optimum (2.5, 2.5), $z = 22.5$ — just round it? $2^2 = 4$ candidates:

candidate	$x + 3y \leq 10$	$3x + y \leq 10$	$z = 4x + 5y$
(2, 2)	8 ✓	8 ✓	18
(3, 2)	9 ✓	11 ×	infeasible
(2, 3)	11 ×	—	infeasible
(3, 3)	12 ×	—	infeasible

but: (1, 3): 10 ✓, 6 ✓, $z = 19$

$$\underbrace{z_{\text{LP}} = 22.5}_{\text{bound}} \geq \underbrace{z_{\text{IP}} = 19}_{\text{optimum, at (1,3)}} > \underbrace{18}_{\text{best rounding}}$$

KEY 1. rounding loses *feasibility* and *optimality* — in n dims, all 2^n candidates can die.

2. integrality changes *where* the answer is.

3. but $z_{\text{LP}} \geq z_{\text{IP}}$ *always* — deleting a constraint only enlarges the feasible set.

demo `demo1.toy_ip.py` — drag c_2 : the IP optimum *hops*; no rounding rule predicts the hops.

takeaway the relaxation gives not the answer but a *bound* — and bounds are how every IP is actually solved.

3. Knapsack — play it, then formulate it

demo `ip-games / knapsack` — play FIRST: rucksack, 21 kg, six items. <https://hankpark0706.github.io/ip-games/knapsack.html>

item	rifle	boots	helmet	canteen	med kit	rations	
weight (kg)	5	7	8	8	8	2	$W = 21$
value (pts)	9	8	9	8	3	9	

$x_j = 1$ if item j packed, 0 if not — a *choice*, not a quantity

$$\begin{aligned} \max \quad & \sum_j v_j x_j \\ \text{s.t.} \quad & \sum_j w_j x_j \leq W \\ & x_j \in \{0, 1\} \quad \forall j \end{aligned}$$

all variables binary $\Rightarrow$ a **binary program (BP)**

greedy (value/weight): rations, rifle, boots = 26 (14 kg)

optimum: rations, rifle, helmet = **27** (15 kg)

$$\underbrace{33.875}_{\text{LP (bound)}} \geq \underbrace{27}_{\text{IP}} > \underbrace{26}_{\text{greedy and rounded LP}}$$

KEY one constraint, six variables — already NP-hard.

takeaway the optimum uses 15 kg of 21 — integrality can leave capacity on the table.

4. From scalar model to matrix form — the rewriting dictionary

modeling = scalar lines; solving = matrices: Gurobi receives exactly (c, h, A, G, b)

first, one with numbers — watch every move:

$$\begin{aligned} \min \quad & x_1 - 2x_2 & \quad & - \max \quad -x_1 + 2u - 2v \\ \text{s.t.} \quad & x_1 + x_2 \geq 3 & \quad & \text{s.t.} \quad -x_1 - u + v \leq -3 \\ & x_1 - x_2 = 1 & \quad & x_1 - u + v \leq 1, \quad -x_1 + u - v \leq -1 \\ & x_1 \geq 0, \quad x_2 \text{ free} & \quad & x_1, u, v \geq 0 \quad (x_2 = u - v) \end{aligned} \quad \Rightarrow$$

min flipped, $\geq$ flipped, = split into two $\leq$, free x_2 split into $u - v$ — then read off:

$$c = \begin{pmatrix} -1 \\ 2 \\ -2 \end{pmatrix}, \quad A = \begin{pmatrix} -1 & -1 & 1 \\ 1 & -1 & 1 \\ -1 & 1 & -1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{pmatrix}, \quad b = \begin{pmatrix} -3 \\ 1 \\ -1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad (\text{last three rows are } x_1, u, v \geq 0)$$

the general dictionary:

you wrote	rewrite as	cost
$\min c^\top x$	$-\max(-c)^\top x$	—
$a^\top x \geq \beta$	$-a^\top x \leq -\beta$	—
$a^\top x = \beta$	$a^\top x \leq \beta$ and $-a^\top x \leq -\beta$	one row
x_j free	$x_j = x_j^+ - x_j^-$, $x_j^+, x_j^- \geq 0$	one variable
bound $x_j \leq u_j$	one more row	one row
$x_j \in \{0, 1\}$	$x_j \in \mathbb{Z}$, $0 \leq x_j \leq 1$	—
$a^\top x \leq \beta$, want =	$a^\top x + s = \beta$, $s \geq 0$ (<i>slack</i>)	one variable
$a^\top x \geq \beta$, want =	$a^\top x - s = \beta$, $s \geq 0$ (<i>surplus</i>)	one variable

slack = the leftover, given a name watch: a split *integer* variable needs **both** halves integer

KEY scalar form for modeling, matrix form for solving — the *same problem*. “is it a MILP?” is checked by reduction.

5. Epigraph trick — the objective becomes a variable

first, one with numbers — three crews, mission ends when the *slowest* finishes. make the worst case a variable t :

$$\min_x \max\{3x_1 + x_2, x_1 + 4x_2, 2x_1 + 3x_2\} \implies \min_{x,t} t$$

$$\text{s.t. } 3x_1 + x_2 \leq t, \quad x_1 + 4x_2 \leq t, \quad 2x_1 + 3x_2 \leq t$$

in general, worst-case objectives turn linear:

$$\min_x \max\{f_1(x), \dots, f_k(x)\} \iff \min_{x,t} t$$

$$\text{s.t. } f_i(x) \leq t \quad \forall i \in [k]$$

t sits above every f_i ; min presses it onto the largest $\Rightarrow$ optimal t = the max. linear $f_i \Rightarrow$ an LP

demo demo2d / demo2e.cvxpy.py — the four-line cvxpy pattern, typed three times: toy LP, toy IP, matching.

KEY “minimize the worst case” became linear rows — the same move returns in weeks 9 and 13.

6. Fixed costs — paying to switch something on

open a site ($x \in \{0, 1\}$, fixed cost c) before stocking $y \geq 0$ (cost f /unit):

$$\text{cost} = cx + fy, \quad y \leq Mx \quad (\text{linking constraint, } M = \text{“big } M\text{”})$$

$$x = 0 \Rightarrow y = 0 \quad \text{closed site holds nothing}$$

$$x = 1 \Rightarrow y \leq M \quad \text{open site holds up to } M$$

KEY “ $y > 0$ ” is not linear — no LP can see it. the binary is the price of a discontinuity: x lets linear constraints say *if-then*.

7. Logical conditions — binaries as switches

yes/no $\leftrightarrow$ yes/no: plain inequalities, no M — check by trying the 0/1 cases:

“if 1 opens, 2 must open”	$x_1 \leq x_2$
“3 or 4, not both”	$x_3 + x_4 \leq 1$
“at most p of n ”	$\sum_i x_i \leq p$
“at least p of n ”	$\sum_i x_i \geq p$
“if 1, then 2 or 3”	$x_1 \leq x_2 + x_3$

yes/no $\leftrightarrow$ *continuous* constraint: **active** at $\bar{y}$: $a^\top \bar{y} = \beta$; **redundant**: deleting it changes nothing. inflate the right side — $M \geq \max_y(a^\top y - \beta)$ — and any constraint goes redundant

redundant = switched **off** $\Rightarrow M(1 - x)$ makes x a **switch**:

$$a^\top y \leq \beta + M(1 - x) \quad \begin{cases} x = 1 : & a^\top y \leq \beta \quad (\text{enforced}) \\ x = 0 : & \text{redundant — off} \end{cases}$$

either-or (a union — not convex, no LP describes it):

$$a_1^\top y \leq b_1 + M(1 - x), \quad a_2^\top y \leq b_2 + Mx, \quad x \in \{0, 1\}$$

at least k of m : one switch each, k switches on:

$$a_i^\top y \leq b_i + M(1 - x_i) \quad \forall i \in [m], \quad \sum_i x_i \geq k$$

minimum load airlift y_1 ($0 \leq y_1 \leq 80$) flies in bulk or not at all: $y_1 = 0$ OR $y_1 \geq 50$

write it in the either-or form — one switch:

$$y_1 \leq 0 + M(1 - x), \quad -y_1 \leq -50 + Mx, \quad x \in \{0, 1\} \quad (M = 80 \text{ works: } y_1 \leq 80)$$

check: $x = 1$: $y_1 \leq 0$ enforced, the ≥ 50 row off; $x = 0$: $y_1 \geq 50$ enforced, the ≤ 0 row off

2 of 3 criteria stockpile (y_1 fuel, y_2 ammo, $y_1 + y_2 \leq 50$); certify if 2 of: total $y_1 + y_2 \geq 40$, fuel $y_1 \geq 25$, ammo $y_2 \geq 25$

flip each $\geq$ to $\leq$, one switch per criterion, at least $k = 2$ on:

$$-y_1 - y_2 \leq -40 + M(1 - x_1), \quad -y_1 \leq -25 + M(1 - x_2), \quad -y_2 \leq -25 + M(1 - x_3)$$

$$x_1 + x_2 + x_3 \geq 2, \quad x \in \{0, 1\}^3 \quad (M = 40, 25, 25 \text{ suffice: with } y \geq 0 \text{ each “off” row is redundant})$$

check: $x = (1, 1, 0)$: total and fuel enforced, the ammo row off — any two on $\Rightarrow$ certified

demo `demo2c_logical_constraints.py` — “at least k of 3” on a box — the feasible set is visibly a non-convex union.

KEY one idea: **big- M makes a constraint redundant on demand — a binary becomes a switch. either-or, k -of- m , if-then: the same switch, different wiring.**

8. How big should M be?

too small: WRONG. the “off” constraint still cuts feasible plans — the solver silently returns the best plan of the wrong problem

too large: WEAK. $x = 0.01$, $M = 10^5$ already unlocks $y \leq 1000$ — bound far from truth, solver slow

the craft: the **smallest certainly-valid** M — a physical quantity (budget, demand, capacity), never 10^6

demo `demo2_sparse_regression.py` — drag M down: coefficients pin against $\pm M$ — a silent modeling bug, live.

9. Capstone — binary variables doing machine learning

the same link doing *feature selection*: $x_j \in \{0, 1\}$ = “feature j used”; penalize the count:

$$\min_{\beta, x} \quad \frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_j \beta_j x_{ij} \right)^2 + \lambda \sum_j x_j$$

$$\text{s.t.} \quad -Mx_j \leq \beta_j \leq Mx_j, \quad x_j \in \{0, 1\}$$

$x_j = 0 \Rightarrow \beta_j = 0$; $x_j = 1 \Rightarrow \beta_j \in [-M, M]$. λ = the fixed charge, per feature

best-subset selection (ℓ_0) — the honest version of what lasso approximates

demo `demo2b_sparse_linear_regression.py` — 8 features, 3 real: raise λ , watch features switch off; a good λ recovers exactly the true 3.

takeaway one trick, three costumes: $y \leq Mx$, $a^\top y \leq b + M(1 - x)$, $-Mx_j \leq \beta_j \leq Mx_j$. choose M as small as validity allows.

10. Matching — a model born canonical

3 pilots, 3 aircraft, c_{ij} = value of pairing (i, j) . the decision is a *table*:

$$X \in \{0, 1\}^{3 \times 3}, \quad x_{ij} = 1 \text{ iff pilot } i \text{ flies aircraft } j$$

rows sum to 1 (each pilot once), columns sum to 1 (each aircraft once):

$$\max_X \quad \langle C, X \rangle$$

$$\text{s.t.} \quad X\mathbf{1} = \mathbf{1}, \quad X^\top \mathbf{1} = \mathbf{1}, \quad X \geq 0$$

read the symbols: $\langle C, X \rangle = \sum_{i,j} c_{ij} x_{ij}$, $\mathbf{1}$ = all-ones vector

matrix form by **flattening**: $x = \text{vec}(X) \in \mathbb{R}^9$, $c = \text{vec}(C) \Rightarrow$ a 6×9 matrix of 0/1, $b = \mathbf{1}$

- each column of A : exactly **two ones** (an *incidence matrix*)
- binaries already gone: row sums = 1 with $x \geq 0$ cap every variable at 1
- rank $A = 5$: any one row is implied by the rest

demo `demo2d (ex. 3)`, `demo3_assignment_matching.py` — solve with *no* integrality declared — the answer is a permutation matrix, max fractionality = 0 every time.

takeaway matching’s LP returns whole numbers with *no* integrality declared. the reason has a name — *total unimodularity* — next lecture.

next lecture assignment, transportation — and *total unimodularity*: why matching's LP came back integer. **Homework 1 is out** (due week 4).